Design a Live Comment System

Linyun Wei, May 9 2025

Functional Requirements

- Viewers can post comments
- Comment Moderation
- Viewers can see all comments posted in near real time
- Viewers can see all comments posted before they join

Scale:

- Assumption:
  - 1. Every stream video lasts for 1 hour.
  - 2. 100,000 concurrent stream videos at any time.
  - 3. For each video, 1000 viewers watch concurrently.
  - 4. Each video has 1000 comments in total.
- Therefore,
  - **30,000 comments/sec** on the entire platform.
    - 1000 comments in an hr / 3600 secs in an hr = 0.3 comment per second per video
    - 0.3 comments per seconds per video * 100K videos
  - **100,000,000 (100M) concurrent viewers** at any time on the entire platform.
    - 100K videos * 1K viewers per video

Non-Functional Requirements

- Assumption: 100M viewers concurrently
- QPS: 30,000 comments per second on the entire platform
- Availability > Consistency for comment creation
- Low latency comment broadcast (~200ms)

API:

Viewers to post a comment:

```
POST /comments/:liveVideoId
{
        Comment
}
```

Viewers can see all comments posted in near real time:

```
GET /comments/:liveVideoId?cursor={last_comment_id}&direction={after|before}&pageSize=50
```

Database:

- Comments

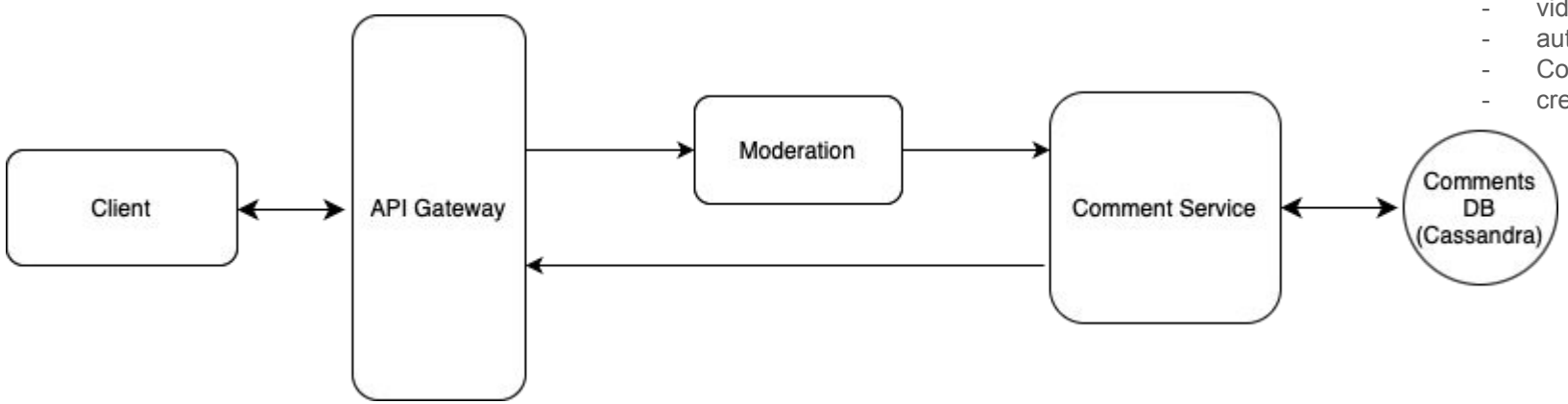
Field	Type	Description
comment_id	String	Primary Key
video_id	String	Foreign Key
user_id	String	Who posted it
content	Text	The actual comment
timestamp	Datetime	When it was posted

Architectural diagram: Functional Requirements

- Viewers can post comments
- Comments must be moderated
- Viewers can see all comments posted in near real time
- Viewers can see all comments posted before they join

Comments

- commentId
- videoId,
- authorId
- Content
- createdAt



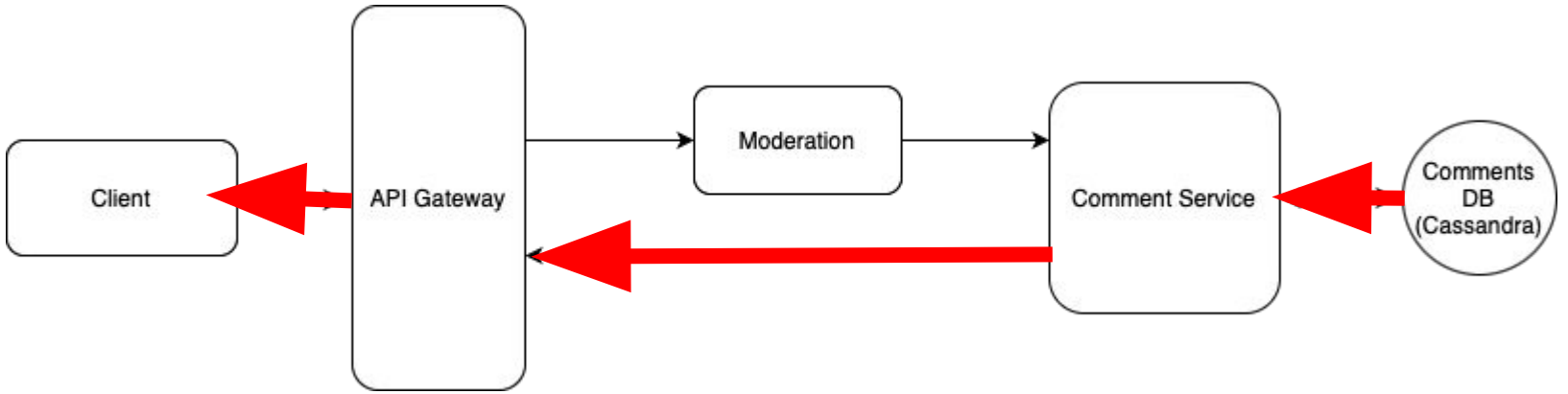
Architectural diagram: Functional Requirements

Historical Comment Fetch:

- When a user first join, they can fetch previous comments using the GET /comments/:videoId API.

Real-Time Comment Update:

- Every 5 secs, the user use GET /comments/:videoId API to fetch the latest comment



Architectural diagram: Functional Requirements

Post Comments:

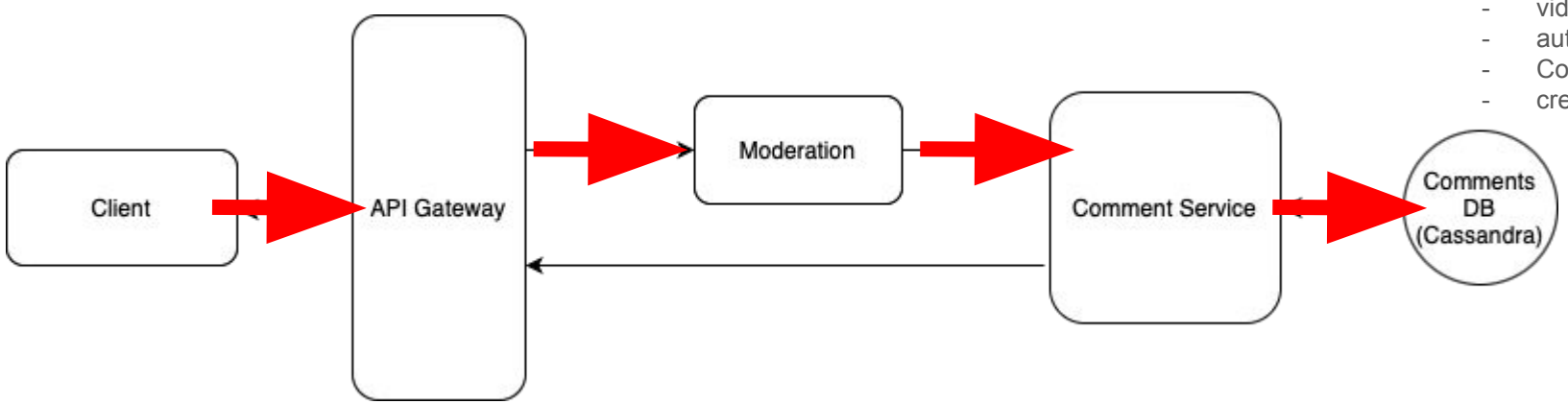
- The user use POST /comments/:liveVideoId API to post a comment

Moderator:

- A Moderation component, which can filter spam

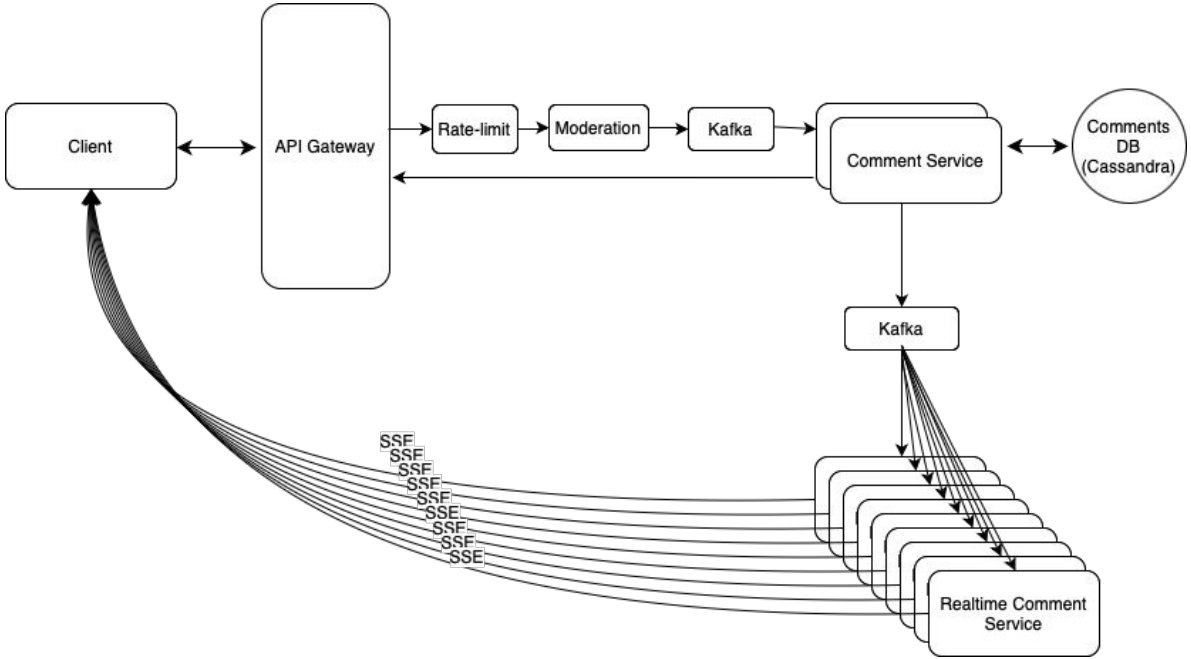
Comments

- commentId
- videoId,
- authorId
- Content
- createdAt



Architectural diagram: Non-Functional Requirements

- Assumption: 100M viewers concurrently
- QPS: 30,000 comments per second on the entire platform
- Availability > Consistency for comment creation
- Low latency comment broadcast (~200ms)



Comments

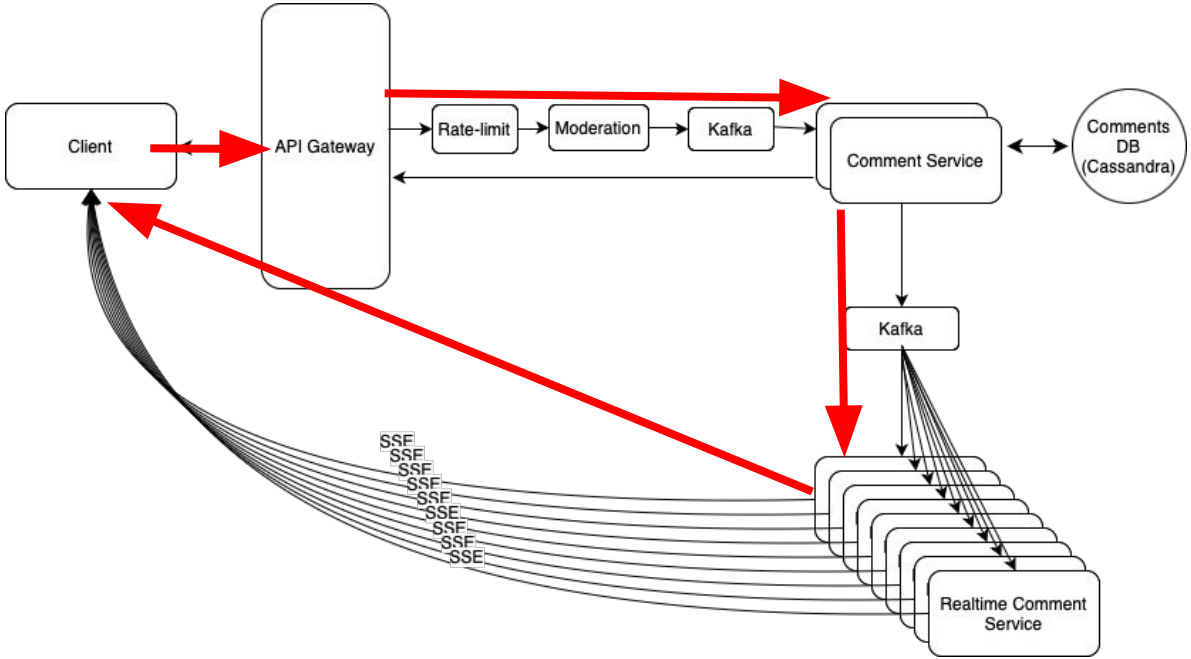
- commentId
- videoId,
- authorId
- Content
- createdAt

Each Realtime Comment Service keeps track of Videos' Id and their viewers' Id

```
{
        videoId1: [user1, user3, user5],
        videoId2: [user2, user4],
        videoId3: [user6]
}
```

Architectural diagram: Non-Functional Requirements

- Assumption: 100M viewers concurrently
- QPS: 30,000 comments per second on the entire platform
- Availability > Consistency for comment creation
- Low latency comment broadcast (~200ms)



Comments

- commentId
- videoId,
- authorId
- Content
- createdAt

Each Realtime Comment Service keeps track of Videos' Id and their viewers' Id

```
{
        videoId1: [user1, user3, user5],
        videoId2: [user2, user4],
        videoId3: [user6]
}
```